

Adaptive Best-of- N with Unknown Reward and Unknown Generation-Cost Distributions

Abstract

We study adaptive generation when neither the prompt-specific reward distribution nor the cost of the next response is known. Opening a box reveals a reward and a response length, and the objective is the best revealed reward minus the total realized generation cost. We first formulate the i.i.d. Pandora problem with random opening costs. Its known-distribution rule uses the mean random cost in the usual fair-cap equation. We then state the learned-threshold guarantee of *Learning When to Stop* for this setting; the proof is unchanged after the constant cost is replaced by the mean random cost and the fair-cap estimator is allowed to use both rewards and costs.

For language-model tasks, the main practical difficulty is putting reward and cost into comparable units. We give a detailed account of that calibration. Alignment uses a Bradley–Terry probability against a prompt-specific high-quality benchmark, while coding uses a train-only isotonic estimate of the probability of correctness. The simplified alignment rule fits its upper tail entirely from the current prompt and needs no training stage. The coding rule learns one global upper tail after isotonic calibration; the same configuration is used at every character cost. Both estimate the next response length by its running mean and generate once more only when estimated gain exceeds estimated character cost. We give one common blueprint and exact descriptions of these two rules.

The experiments compare adaptive stopping with deployable and oracle fixed policies. Alignment uses a train-selected Fixed- N , a test-oracle Fixed- N , and a stronger test-oracle character-budget allocation that knows each prompt’s mean response length. Coding uses train-selected and test-oracle Fixed- N baselines and a fixed character budget selected on the training half and transferred unchanged to test. The Alignment section reports utility, head-to-head win rate at exactly matched expected character cost, and target quality; the Coding section reports utility and target accuracy. In both target experiments, the Fixed- N comparator is selected on the training half and frozen for test. Target calibration plots are shown beside character savings so that train-to-test overshoot or undershoot is visible. We additionally give post-hoc Fixed- N diagnostics at exactly the adaptive test quality, separating allocation efficiency from calibration error. All reported costs are exact cumulative output characters, including for policies whose stopping decisions use estimated costs.

1 Pandora’s box with unknown rewards and unknown costs

We begin with the clean stopping problem. A single type of box is available repeatedly. Opening box i reveals a reward X_i and incurs a nonnegative cost C_i . The pairs

$$(X_1, C_1), (X_2, C_2), \dots \quad \text{are independent and identically distributed,} \quad (1)$$

but their joint distribution is unknown. Reward and cost may be dependent within a pair. Before opening box i , neither X_i nor C_i is known. Once the box is opened, both are observed and its cost is paid. The decision after each opening is whether to stop or to open one more box.

Let

$$M_n = \max_{1 \leq i \leq n} X_i \quad (2)$$

be the best observed reward. A stopping time $T \geq 1$ receives payoff

$$R_T = M_T - \sum_{i=1}^T C_i. \quad (3)$$

We assume $\mathbb{E}[X^+] < \infty$ and $\bar{c} = \mathbb{E}[C] \in (0, \infty)$. Define

$$L(s) = \mathbb{E}[(X - s)_+], \quad S(s) = \Pr(X \geq s), \quad \tau = \sup\{s : L(s) \geq \bar{c}\}. \quad (4)$$

The function L measures the value of one more reward draw above a current best value s . We call τ the fair cap. For the continuous distributions considered here, $L(\tau) = \bar{c}$.

If the joint distribution were known, the optimal rule would stop at

$$T_W = \inf\{n \geq 1 : M_n \geq \tau\}. \quad (5)$$

This is the usual i.i.d. Pandora rule with the deterministic cost replaced by the mean random cost. To see why this replacement is sufficient, condition on the observations before the next opening. The expected change from one more box at incumbent m is

$$\mathbb{E}[(X - m)_+ - C] = L(m) - \bar{c}. \quad (6)$$

No independence between X and C inside a box is needed for this identity. The stopping decision is made before either member of the next pair is observed, and the cost enters additively. Under the standard integrability conditions, the known-distribution expected payoff is $\mathbb{E}[R_{T_W}] = \tau$.

1.1 The learned-threshold theorem

The main theorem of *Learning When to Stop* extends directly to the model above. The only statistical change is that the fair-cap estimate must now learn both the reward distribution entering L and the mean cost $\bar{c}$. We state the result and do not repeat its proof.

After n observed reward–cost pairs, let $\hat{\tau}_n$ be any estimate of the solution to $L(t) = \bar{c}$. For a nonnegative bonus b_n , define

$$U_n = \hat{\tau}_n + b_n, \quad T = \inf\{n \geq 1 : M_n \geq U_n\}. \quad (7)$$

For deterministic widths $w_1, w_2, \dots$, define the local band cost

$$\Gamma_{\mathcal{D}}(w) = S(\tau) \sum_{n \geq 1} w_n \Pr\{\tau \leq M_n < \tau + w_n\}. \quad (8)$$

It charges an upward threshold error only on histories where that error actually makes the learned rule continue after the oracle would stop.

Theorem 1 (Unknown-reward and unknown-cost reduction). *Suppose there is a scale $a > 0$ such that, for every $n \geq 1$ and $x \geq 0$,*

$$\Pr\left(|\hat{\tau}_n - \tau| > a \left\{ \sqrt{\frac{2x}{n}} + \frac{2x}{n} \right\}\right) \leq 2e^{-x}. \quad (9)$$

Use

$$b_n = a \left\{ 4\sqrt{\frac{\log(en)}{n}} + \frac{16 \log(en)}{n} \right\}, \quad U_n = \hat{\tau}_n + b_n, \quad w_n = 2b_n. \quad (10)$$

If $\Pr(X > \tau) > 0$, the stopping rule in Eq. (7) stops almost surely and

$$\mathbb{E}[R_{T_W} - R_T] \leq \Gamma_{\mathcal{D}}(w) + a. \quad (11)$$

Consequently, whenever the right-hand side is at most $\varepsilon\tau$, the learned rule obtains at least a $(1 - \varepsilon)$ fraction of the oracle payoff.

The theorem is deliberately a reduction. It does not say that every unknown reward and cost distribution is easy. It says that once a structured model gives a concentrated estimate of the fair cap, including the uncertainty in mean cost, the stopping loss is controlled by the estimation scale and by a local decision term. The practical LLM algorithms below use this same crossing principle, but use regularized fitted tails rather than claiming the finite-sample confidence statement in Eq. (9).

2 Putting LLM reward and generation cost on one scale

In a language-model application, opening a box means generating and scoring one response. Its length is not known until generation finishes. Let K_i be the output character count of response i . For a supplied divisor D , we convert it to utility cost

$$C_i = \frac{K_i}{D}. \quad (12)$$

Thus D is the number of characters worth one unit of utility. It is an external economic scale, not a prediction of response length. The random variable K_i is the unknown opening cost. During evaluation we always charge the exact sum $\sum_{i \leq T} K_i/D$; an estimated length is used only to decide whether to generate the next response.

The other side of the comparison is harder. A raw reward-model score has no natural unit relative to characters. It can be negative, its scale changes across reward models, and a one-point increase need not have the same meaning on different prompts. We therefore map the raw score to a bounded, task-relevant utility before comparing the value of another response with K_i/D . Alignment and coding require different maps because their outcomes have different meanings.

2.1 Alignment: Bradley–Terry utility

Let Y be a raw alignment reward and let β be the score of a high-quality response for the same prompt. We use

$$u_\beta(y) = \sigma(y - \beta) = \frac{1}{1 + \exp(\beta - y)}. \quad (13)$$

This is the unit-temperature Bradley–Terry probability that a response with score y beats a response with score β . There are three reasons for this choice.

First, alignment reward models are designed to rank responses according to preference. A pairwise win probability is therefore closer to their intended semantics than treating the raw score itself as cardinal utility. Second, the difference $y - \beta$ removes prompt-specific score offsets. A score of five is not intrinsically good, but being well above a high benchmark for the same prompt is meaningful. Third, the sigmoid bounds utility in $(0, 1)$, which gives a stable and interpretable scale for comparison with character cost.

The benchmark is the prompt’s 99th reward percentile. It is deliberately difficult: $u_\beta(y) = 1/2$ means that the candidate is estimated to tie a response near the top one percent of that prompt’s reward distribution. The online algorithm does not know this percentile. It estimates it from the observed score prefix using a regularized upper-tail model. A full-pool 99th percentile is used only after stopping to evaluate all policies on a common scale. We use a unit-temperature link and do not fit a Bradley–Terry temperature; consequently, the map is best understood as a transparent BT-style utility scale rather than a separately validated probability model.

2.2 Coding: probability of correctness from isotonic regression

For coding, the final reward is binary correctness. The natural utility of a candidate with raw score y is therefore its probability of being correct. On each outer split, we use all labeled generations from the 41 training problems to fit

$$\hat{g} \in \arg \min_{\substack{g: \mathbb{R} \rightarrow [0,1] \\ g \text{ nondecreasing}}} \sum_{i \in \mathcal{A}} \sum_{j=1}^{512} \{Z_{ij} - g(Y_{ij})\}^2, \quad Z_{ij} \in \{0, 1\}. \quad (14)$$

The calibrated utility is $P_{ij} = \hat{g}(Y_{ij})$.

Isotonic regression is appropriate here because the reward score is intended to rank candidate programs, but there is no reason to assume that equal score increments correspond to equal changes in correctness probability. The monotonicity constraint preserves the reward model’s ordering, while the piecewise-constant fit learns the shape of the score–correctness relationship without imposing a sigmoid or choosing reward bins. It also maps directly to the quantity used by the coding objective:

$$\mathbb{E}[Z \mid Y = y] \approx \hat{g}(y). \quad (15)$$

Unlike alignment, coding has observed binary training labels, so a direct probability calibration is preferable to a relative BT comparison.

The isotonic map is fit only on the training half, then frozen. Scores outside the training range receive the corresponding endpoint prediction. The test policy observes a program’s raw reward and length, applies the frozen map, and never observes correctness. Correctness is revealed only to the offline evaluator. Isotonic plateaus can tie several programs; the implementation returns the first opened program attaining the largest calibrated value.

2.3 A common algorithmic blueprint

After calibration, the two applications have the same structure. The policy opens a short prefix and forms an optimistic fitted distribution for the utility of the next response. Alignment forms this distribution from the current

Algorithm 1 Blueprint for adaptive LLM generation

Require: Any task-specific training data; character divisor D ; horizon H ; initial count m_0

- 1: Fit the task-specific calibration and tail information, if required
 - 2: Generate m_0 responses and observe their scores and character counts
 - 3: Map each observed score to the task’s bounded utility scale
 - 4: **for** $n = m_0, m_0 + 1, \dots, H - 1$ **do**
 - 5: Fit the task’s optimistic utility distribution $\hat{F}_n$
 - 6: Let v_n be the largest calibrated utility among the first n responses
 - 7: Set c_n to the observed mean response length divided by D
 - 8: Find the fair cap τ_n from $\mathbb{E}_{\hat{F}_n}[(U - \tau_n)_+] + \varepsilon_n = c_n$ ▷ use the boundary convention in (16)
 - 9: **if** $v_n \geq \tau_n$ **then**
 - 10: Stop and return the best response observed so far
 - 11: **end if**
 - 12: Generate response $n + 1$ and observe its score and exact length
 - 13: **end for**
 - 14: Return the best observed response and charge the exact total character count
-

prefix alone; coding uses one tail learned on training problems. If c_n is the estimated utility cost of one more response, the fitted reservation value, or fair cap, is

$$\tau_n = \inf \left\{ t : \mathbb{E}_{\hat{F}_n}[(U - t)_+] + \varepsilon_n \leq c_n \right\}, \quad (16)$$

where ε_n is the policy’s small optimism correction. The policy stops exactly when its best observed utility is at least τ_n . Thus the fair cap combines the predicted benefit and cost of one more generation into one interpretable threshold.

Algorithm 1 is a single adaptive stopping rule, not a mixture with Fixed- N . Fixed- N appears only as an evaluation baseline. The two concrete policies differ in the calibration map, the fitted upper tail, and a small number of regularization choices described next.

3 Alignment

3.1 Exact alignment policy

Algorithm 2 states the decision at the distribution level. The rule has no training stage. At each prefix it fits a shifted exponential to the inclusive upper half of the current prompt’s exponentiated scores, uses the fitted law to obtain a fair cap, and compares the incumbent with that cap. The confidence multiplier γ is an explicit input; all experiments use its default value 0.6.

Algorithm 2 Alignment: training-free adaptive stopping rule

Require: Divisor D ; horizon $H = 960$; initial count $m_0 = 5$; confidence γ (default 0.6)

```
1: Generate  $m_0$  responses and observe scores and lengths  $(Y_j, K_j)$ 
2:  $n \leftarrow m_0$ 
3: while  $n < H$  do
4:    $E_j \leftarrow \exp\{\text{clip}(Y_j, -50, 50)\}$  for  $j \leq n$ 
5:    $T_n \leftarrow$  the  $\lfloor n/2 \rfloor + 1$  largest observed  $E_j$  ▷ inclusive upper half
6:    $\ell_n \leftarrow \min T_n$  ▷ fitted shift
7:    $\hat{\lambda}_n \leftarrow \max\{\text{mean}(T_n) - \ell_n, 10^{-8}\}^{-1}$  ▷ fitted rate
8:    $\hat{X}_n \sim \text{ShiftedExp}(\ell_n, \hat{\lambda}_n)$ 
9:    $h_n \leftarrow \sqrt{\log(20)/|T_n|}$ 
10:   $s_n^+ \leftarrow \hat{\lambda}_n^{-1}(1 + \gamma h_n)$  ▷ optimistic tail scale
11:   $s_n^- \leftarrow \max\{\hat{\lambda}_n^{-1}(1 - \gamma h_n), 10^{-8}\}$  ▷ conservative benchmark scale
12:   $b_{n,\gamma} \leftarrow \log(\ell_n + s_n^- \log 50)$ 
13:  Define  $\hat{F}_{n,\gamma}$  by: with probability  $1/2$ ,  $U = 0$ ; otherwise draw  $Z \sim \text{Exp}(1)$  and set  $U = \sigma\{\log(\ell_n + s_n^+ Z) - b_{n,\gamma}\}$ 
14:   $c_n \leftarrow (nD)^{-1} \sum_{j=1}^n K_j$  ▷ estimated cost of one more response
15:   $\varepsilon_n \leftarrow 0.002 \sqrt{\log(20)/(2n)}$ 
16:   $\tau_n \leftarrow \inf \left\{ t \in [0, 1] : \mathbb{E}_{U \sim \hat{F}_{n,\gamma}} [(U - t)_+] + \varepsilon_n \leq c_n \right\}$  ▷ fair cap of  $\hat{F}_{n,\gamma}$  at cost  $c_n$ 
17:   $v_n \leftarrow \sigma(\max_{j \leq n} Y_j - b_{n,\gamma})$ 
18:  if  $v_n \geq \tau_n$  then
19:    break ▷ incumbent has reached the fair cap
20:  end if
21:  Generate response  $n + 1$  and observe  $(Y_{n+1}, K_{n+1})$ 
22:   $n \leftarrow n + 1$ 
23: end while
24:  $J \leftarrow \min \arg \max_{j \leq n} Y_j$ 
25: Return response  $J$  and charge exact cost  $\sum_{j=1}^n K_j / D$ 
```

The upper-tail component of $\hat{F}_{n,\gamma}$ has probability 0.5; the remaining probability is placed at zero and contributes nothing to the positive part. The factor $\log 50$ extrapolates from tail mass 0.5 to the implied 99th percentile.

Because the expected positive part decreases with t , the test $v_n \geq \tau_n$ is algebraically identical to evaluating it once at v_n and comparing the result with c_n . The implementation therefore does not need a numerical root finder; τ_n is the cleaner conceptual statement of the same stopping decision.

Five initial responses, default confidence $\gamma = 0.6$, and bonus 0.002 are frozen for every generator, divisor, and final split. There is no reward prior, cost prior, fitted temperature, or other training quantity in the stopping decision. This is one UCB-style Pandora crossing rule: it does not consult Fixed- N , impose a Fixed- N guard, or inspect the next response length before opening it.

3.2 Utility experiment and fixed-policy baselines

We evaluate 100 Alpaca prompts for each of five generators: Gemma-2-9B, Llama-3.1-8B, Llama-3.2-3B, Mistral-7B, and Qwen-2.5-7B. Every prompt has 960 responses scored by RM-Mistral-7B. Ten deterministic splits divide the prompts into 50 training and 50 test prompts. The training half selects the implementable Fixed- N baseline and, in the target experiment, the adaptive divisor. It fits nothing used by Algorithm 2, whose formula is fixed across generators, divisors, and final splits. We replay four permutations per training prompt and eight per test prompt. The evaluated divisors are

$$D \in \{10^5, 5 \times 10^5, 10^6, 5 \times 10^6, 10^7, 5 \times 10^7\}. \quad (17)$$

For evaluation only, prompt x has full-pool benchmark β_x^* , the fixed empirical 99th percentile of its 960 rewards. If a policy returns raw reward y , its evaluated utility is

$$U_x(y, T) = \sigma(y - \beta_x^*) - \frac{1}{D} \sum_{j=1}^T K_j. \quad (18)$$

The online policy never sees β_x^* . It uses b_n from Algorithm 2.

We compare the adaptive rule with three fixed policy classes.

1. Train-selected Fixed- N (implementable). For every split and divisor, choose the integer $N \in \{1, \dots, 960\}$ that maximizes mean utility on the 50 training prompts. Freeze that N and run it on the test prompts. This rule uses no test outcome to choose N .

2. Test-oracle Fixed- N . Choose the single integer N that maximizes mean utility on the test prompts and the same held-out permutations used for evaluation. This is not implementable; it measures the best uniform generation count available after seeing the test results.

3. Test-oracle fixed character budget. This stronger oracle knows each test prompt’s mean full-pool response length

$$\ell_x = \frac{1}{960} \sum_{j=1}^{960} K_{xj}. \quad (19)$$

Given one common character budget R , it allocates

$$N_x(R) = \max \left\{ 1, \min \left\{ 960, \left\lfloor \frac{R}{\ell_x} \right\rfloor \right\} \right\} \quad (20)$$

generations to prompt x . The test set itself chooses R to maximize mean utility. We enumerate the exact allocation breakpoints $R = n\ell_x$, rather than using an approximate grid. Although R determines the allocation, evaluation charges the actual cumulative character counts, not R . This baseline is oracle in two ways: it knows each test prompt’s mean length and tunes the common budget on test outcomes.

For baseline B , we first compute the percentage relative improvement within each prompt split s ,

$$I_{s,B} = 100 \frac{\bar{U}_{\text{adaptive},s} - \bar{U}_{B,s}}{\bar{U}_{B,s}}, \quad (21)$$

and report its mean across splits. This is easier to read than a ratio: zero denotes equality and positive values favor adaptive stopping on this split-normalized metric. Figure 1 shows pointwise 95% Student- t intervals across the ten prompt splits.

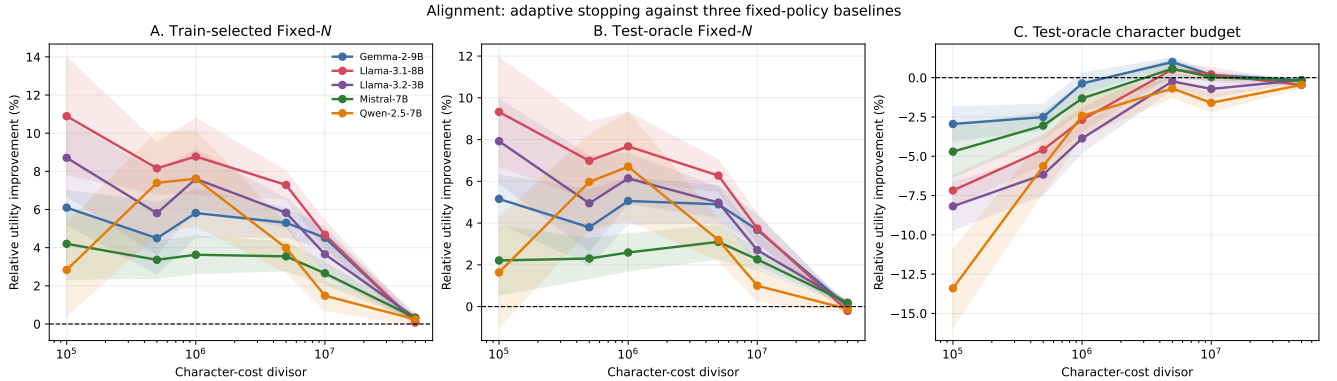


Figure 1: Alignment percentage relative utility improvement over the three requested baselines. The first baseline selects N on the training half; the other two select their parameter on the test half and are intentionally optimistic. The dashed zero line denotes equal utility.

Table 1 averages the six displayed divisor-level percentage improvements for each generator. The training-free rule exceeds train-selected Fixed- N in all 30 generator-divisor cells and the test-oracle Fixed- N in 28 of 30. Averaged equally across the 30 cells, the improvements are 4.65% and 3.81%, respectively. The prompt-specific character-budget oracle is stronger: adaptive utility is 2.37% lower on average and exceeds it in 6 of 30 cells. Relative to the previous training-shrunk rule, the mean utility change is only -0.00040 in absolute units, while the entire alignment training stage disappears.

Generator	Train-selected N	Oracle N	Oracle character budget
Gemma-2-9B	+4.39%	+3.76%	−0.80%
Llama-3.1-8B	+6.65%	+5.63%	−2.36%
Llama-3.2-3B	+5.32%	+4.48%	−3.22%
Mistral-7B	+2.95%	+2.11%	−1.45%
Qwen-2.5-7B	+3.93%	+3.06%	−4.03%
All generator–divisor cells	+4.65%	+3.81%	−2.37%

Table 1: Alignment percentage relative utility improvement, averaged equally over the six displayed character divisors.

3.3 Win rate at matched expected characters

Utility combines quality and cost, so a utility gain alone does not reveal whether adaptive stopping returns better responses, spends fewer characters, or both. For each generator, split, and divisor, we therefore replay Fixed- N independently. On the test set, a randomized mixture of the two adjacent integer values of N is chosen so that its expected character count equals the adaptive policy’s test character count exactly. We then compare the two returned raw rewards head to head, with a tie contributing one half. Because the mixing weight uses test cost after the fact, this is an oracle diagnostic rather than a deployable baseline.

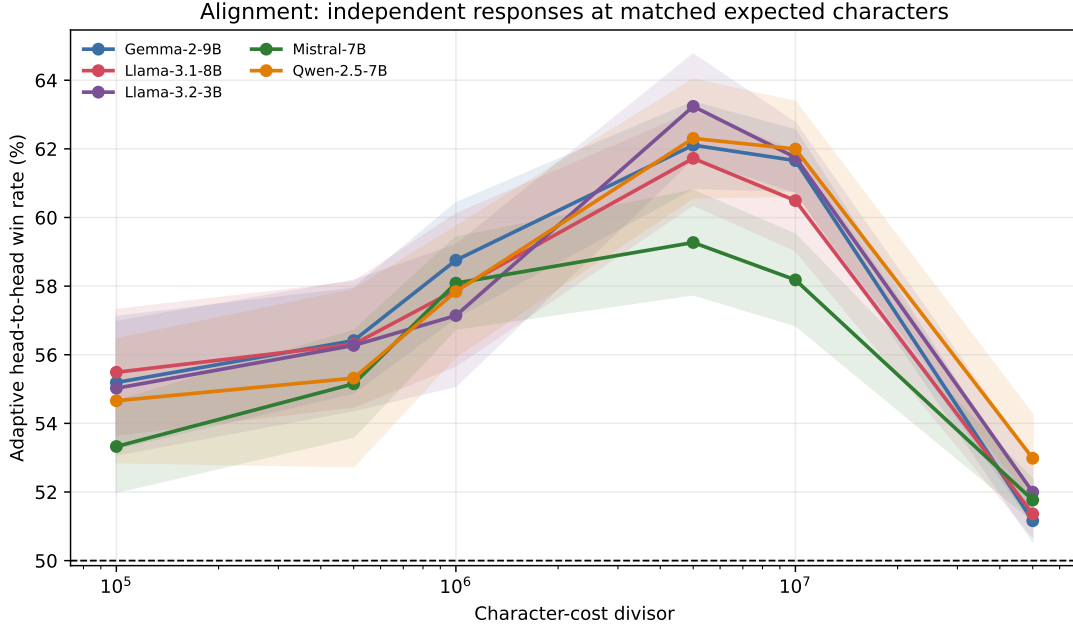


Figure 2: Alignment head-to-head win rate against independent Fixed- N responses at exactly matched expected characters. Bands are pointwise 95% Student- t intervals over ten splits; 50% denotes equality.

Across the 30 generator–divisor cells, mean adaptive win rates range from 51.16% to 63.24%. The lower endpoint of every displayed 95% interval is above 50%. Thus the utility improvement over uniform Fixed- N is not only an artifact of spending fewer characters: with expected cost held fixed, adaptive allocation selects the better response more often.

3.4 Target quality and character saving

Here target quality means the test average of the BT evaluation utility in Eq. (18) before subtracting cost. For each outer training half, generator, and target in $\{0.30, 0.35, \dots, 0.60\}$, we make two independent selections. Among the eleven adaptive divisors from 5×10^4 through 5×10^7 , we select the single policy with the lowest training mean characters that reaches the target. Among all integer $N \in \{1, \dots, 960\}$, we likewise select the lowest-character

Fixed- N reaching the target. Both choices are frozen and run on the other half. There is no safety margin, interpolation, or policy randomization in either deployed rule.

The train-to-test Fixed- N comparison is retained in the audit tables. Figure 3 reports the equal-quality oracle saving and the adaptive rule’s calibration to the requested target. After observing the test curves, the oracle minimizes expected characters over all non-adaptive distributions on $N \in \{1, \dots, 960\}$ subject to expected quality equaling the adaptive rule’s realized test quality. This is a linear program with two equality constraints, so an optimum uses at most two counts. Their random choice is made before generation and never reads a response prefix.

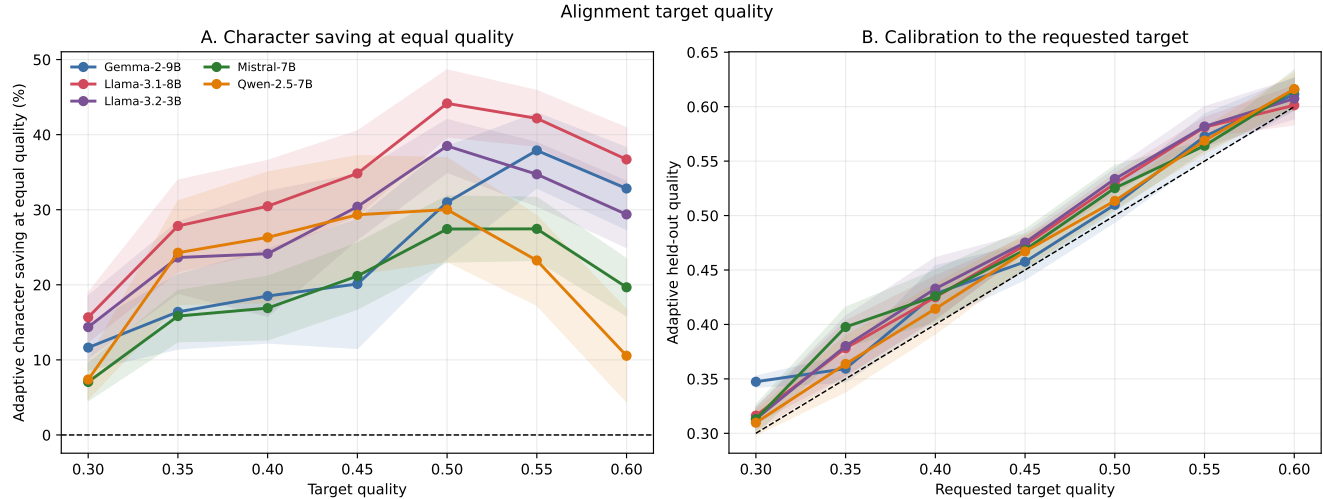


Figure 3: Alignment target-quality results. Left: adaptive character saving against the post-hoc minimum-cost non-adaptive Fixed- N oracle subject to exactly the adaptive rule’s expected held-out quality. The oracle randomizes over at most two fixed counts before generation. Right: adaptive held-out quality versus the requested target; the diagonal denotes exact calibration. Bands are 95% intervals across ten splits.

The right panel shows the target-transfer error directly; the adaptive rule’s mean absolute held-out deviation from the requested target is 2.49 percentage points. The equal-quality diagnostic in the left panel removes that mismatch from the cost comparison. Averaged over the five generators, adaptive character savings at targets 0.30 through 0.60 are respectively 11.22%, 21.59%, 23.26%, 27.17%, 34.22%, 33.10%, and 25.82%. All 35 generator–target cells are positive, with a 25.20% mean saving across cells. The matched expected quality agrees with adaptive quality to numerical precision in every one of the 350 split-level comparisons. These larger, uniformly positive values isolate adaptive allocation efficiency, but they are oracle diagnostics because their two Fixed- N endpoints and mixing weight minimize character cost on the test curve.

4 Coding

4.1 Exact coding policy

The coding rule uses the same crossing logic, but it has a separate learning procedure because correctness labels are available on the training problems. That procedure returns a monotone score-to-correctness map and one global shifted-exponential upper-tail model. The stopping procedure freezes both objects, maps each observed test score to calibrated probability, and finds a fair cap under an optimistic version of the learned tail. There is no online local-tail fit and no divisor-specific configuration table.

Algorithm 3 Coding: learn the calibrated law, then stop at its fair cap

```
1: procedure LEARNCODINGLAW( $\{(Y_{ij}, Z_{ij}) : i \in \mathcal{A}, j \leq 512\}$ )
2:   Fit the monotone calibration map  $\hat{g}$  in Eq. (14)
3:   Apply  $\hat{g}$  to the training scores to obtain calibrated correctness probabilities  $P_{ij} = \hat{g}(Y_{ij})$ 
4:   For each training problem, fit a shifted exponential to the inclusive upper quarter of its calibrated probabilities
5:   Average the fitted shifts and scales to obtain the global tail  $\hat{F}^{\text{tail}} = \text{ShiftedExp}(\bar{\ell}, \bar{s})$ 
6:   return the frozen calibration  $\hat{g}$  and tail parameters  $(\bar{\ell}, \bar{s})$ 
7: end procedure

8: procedure STOPCODING( $\hat{g}, \bar{\ell}, \bar{s}, D_{\text{res}}$ )
Require: Horizon  $H = 512$ ; initial count  $m_0 = 3$ ; confidence  $\gamma$  (default 0.8); confidence level  $\delta$  (default 0.05)
9:   Generate  $m_0$  programs, observe  $(Y_j, K_j)$ , and set  $P_j \leftarrow \hat{g}(Y_j)$  for  $j \leq m_0$ 
10:   $n \leftarrow m_0$ 
11:  while  $n < H$  do
12:     $v_n \leftarrow \max_{j \leq n} P_j$  ▷ best calibrated correctness probability
13:     $m_n \leftarrow |\{j \leq n : P_j \geq Q_{0.75}(P_1, \dots, P_n)\}|$  ▷ observed upper-tail count
14:     $h_n \leftarrow \sqrt{\log(1/\delta)/m_n}$ 
15:     $\ell_n^+ \leftarrow \text{clip}_{[0,1]}(\bar{\ell} + \gamma \bar{s} h_n)$  ▷ optimistic tail shift
16:     $s_n^+ \leftarrow \bar{s}(1 + \gamma h_n)$  ▷ optimistic tail scale
17:     $\rho_n \leftarrow 0.25(m_0/n)$  ▷ regularized upper-tail mass
18:    Define  $\hat{F}_n$  with probability  $\rho_n$ , draw  $U \sim \text{ShiftedExp}(\ell_n^+, s_n^+)$  conditioned on  $U \leq 1$ 
    Otherwise set  $U = 0$ 
19:     $c_n \leftarrow (nD_{\text{res}})^{-1} \sum_{j=1}^n K_j$  ▷ estimated cost of one more program
20:     $\tau_n \leftarrow \inf \{t \in [0, 1] : \mathbb{E}_{U \sim \hat{F}_n}[(U - t)_+] \leq c_n\}$  ▷ fair cap of the calibrated law
21:    if  $v_n \geq \tau_n$  then
22:      break ▷ incumbent has reached the fair cap
23:    end if
24:    Generate program  $n + 1$ , observe  $(Y_{n+1}, K_{n+1})$ , and set  $P_{n+1} \leftarrow \hat{g}(Y_{n+1})$ 
25:     $n \leftarrow n + 1$ 
26:  end while
27:   $J \leftarrow \min \arg \max_{j \leq n} P_j$ 
28:  return program  $J$  and the exact opened character count  $\sum_{j=1}^n K_j$ 
29: end procedure
```

Computing the coding fair cap. The upper-quarter exponential fit in the learning procedure is exact shorthand for $\ell_i = Q_{0.75}(P_{i1}, \dots, P_{iH})$ and $s_i = \max\{\text{mean}_{j: P_{ij} \geq \ell_i}(P_{ij} - \ell_i), 10^{-8}\}$, followed by $\bar{\ell} = \text{mean}_i \ell_i$ and $\bar{s} = \text{mean}_i s_i$. The mass $\rho_n = 0.25(3/n)$ incorporates both the fitted upper-quarter mass and the implementation’s smooth opportunity-decay factor. Consequently the fair-cap test is algebraically identical to the former expected-gain calculation; the 32-point quadrature is only how the implementation evaluates the expectation, not an additional algorithmic decision.

For utility, one development-selected multiplier sets $D_{\text{res}} = 0.85D$ at every economic divisor D . This changes only the reservation comparison; evaluation always records binary correctness minus the exact opened character count divided by D . For target accuracy, D_{res} is the frozen target-specific divisor described below and the reported outcome is correctness together with exact characters. Thus the old ten-row divisor-specific configuration table is replaced by the two procedures in Algorithm 3. The only terminal cap is the absolute 512-program data boundary.

4.2 Utility experiment and fixed-policy baselines

We evaluate ten final 41/42 splits, with 48 random permutations for every test problem. Each candidate pool contains 512 programs. For each split, the isotonic map and global tail use only the 41 training problems. The adaptive configuration and D_{res} for every economic divisor were frozen before final split IDs 75–84. Development split IDs 63–67 selected the single multiplier $D_{\text{res}}/D = 0.85$; no final split selected a tail model or divisor-specific parameter. The final evaluation uses binary correctness minus exact cumulative characters divided by D .

We compare against three baselines.

1. Train-selected Fixed- N (implementable). For each outer split and divisor, choose the integer N that maximizes mean correctness minus character cost on the 41 training problems. Freeze N and evaluate it on the 42 test problems with the same permutations used for the adaptive policy.

2. Test-oracle Fixed- N . Choose the integer $N \in \{1, \dots, 512\}$ that maximizes the same utility on the current test problems and permutations. This is a post-hoc upper envelope for uniform Fixed- N , not a deployable method.

3. Train-selected fixed character budget (implementable). For each split and divisor, use 48 deterministic permutations of the 41 training problems to select one global threshold R . The policy opens one program and continues while the cumulative characters already observed are at most R . We choose R to maximize training mean correctness minus exact character cost, enumerating every observed training-prefix breakpoint so the optimization is exact. We then freeze R and apply the same stopping rule to the 42 test problems. The rule neither predicts the next program length nor reads test outcomes; because the final program is charged in full, its realized total may exceed R by that program’s length.

Figure 4 and Table 2 use the percentage relative improvement in Eq. (21). The mean splitwise improvement over the train-selected baseline is positive at every displayed divisor and averages 2.63%. Against test-oracle Fixed- N , it averages -0.12% : the mean is positive at five divisors and negative at five. This is the more demanding comparison because the oracle selects N after seeing evaluation outcomes, whereas adaptive stopping changes its stopping time problem by problem using only the observed prefix. Relative to the train-selected fixed character budget, adaptive utility improves by 3.48% on average. Its mean improvement is positive at all ten divisors, and six of the ten 95% intervals lie entirely above zero.

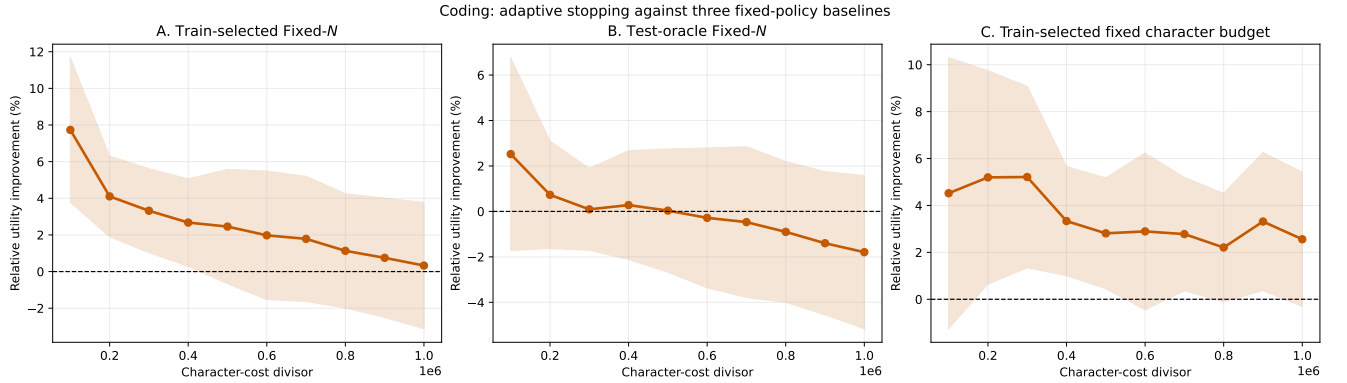


Figure 4: Coding percentage relative utility improvement over train-selected Fixed- N , test-oracle Fixed- N , and the train-selected fixed character budget. Bands are pointwise 95% Student- t intervals over ten final problem splits; the dashed zero line denotes equal utility.

Character divisor D	Train-selected N	Test-oracle N	Fixed char. budget
100,000	+7.73%	+2.52%	+4.52%
200,000	+4.10%	+0.73%	+5.20%
300,000	+3.32%	+0.09%	+5.21%
400,000	+2.68%	+0.28%	+3.34%
500,000	+2.46%	+0.04%	+2.81%
600,000	+1.98%	−0.28%	+2.89%
700,000	+1.79%	−0.47%	+2.78%
800,000	+1.13%	−0.90%	+2.21%
900,000	+0.75%	−1.40%	+3.31%
1,000,000	+0.33%	−1.79%	+2.56%
Mean	+2.63%	−0.12%	+3.48%

Table 2: Coding percentage relative utility improvement over ten final splits. The fixed character budget is selected only on the training half and then frozen for test.

4.3 Target accuracy and character saving

For each 41/42 split and requested accuracy in $\{0.25, 0.30, 0.35\}$, the isotonic map is fit on the 41 training problems. The Fixed- N baseline is the minimum-character integer N that reaches the requested accuracy on that same training half; that N is then frozen for test. The adaptive target selector uses exactly Algorithm 3 but replaces $0.85D$ by one reservation divisor chosen from $\{25,000, 35,000, \dots, 10,000,000\}$. Development split IDs 55–59 freeze the mean held-out accuracy and geometric-mean character cost at each divisor. For a requested accuracy, the

selector takes the cheapest divisor whose development accuracy reaches the target: 25,000, 140,000, and 2,800,000 for the three reported targets. There is no current-split blend, configuration search, safety margin, interpolation, or policy mixture in the adaptive rule. The outer 41-problem training half is still the only source for its isotonic map and global tail.

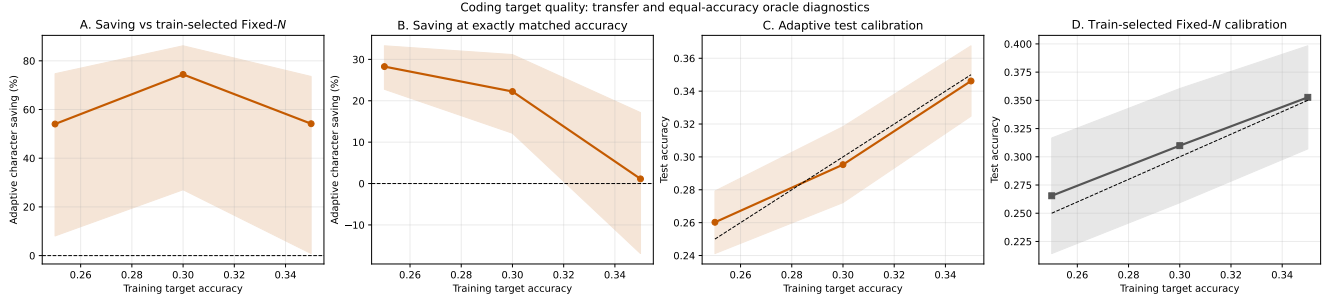


Figure 5: Coding target-accuracy results. A: aggregate adaptive character saving relative to train-selected Fixed- N . B: saving relative to the post-hoc non-adaptive Fixed- N randomization with exactly the adaptive aggregate test accuracy. C–D: held-out calibration of the adaptive and train-selected Fixed- N rules. The diagonal is exact target transfer; bands are 95% intervals across ten splits.

At targets 0.25, 0.30, and 0.35, adaptive test accuracies are 0.260, 0.295, and 0.346, while train-selected Fixed- N obtains 0.265, 0.310, and 0.353. Aggregate character savings are 54.04% (95% interval 8.06–74.71%), 74.41% (27.04–86.19%), and 54.17% (0.83–73.58%). Thus the simplest selector overshoots the first target by 1.02 points and undershoots the latter two by 0.47 and 0.38 points. The calibration panels remain essential: cost savings are results for policies selected before test, not a claim that their realized test accuracies are identical.

For the equal-accuracy diagnostic, we first average the exact held-out Fixed- N accuracy and character curves over the ten final splits. We then choose the minimum-character non-adaptive rule whose average accuracy equals the adaptive average. Because an integer N will almost never hit a decimal accuracy exactly, this rule randomizes between two adjacent counts before any program is generated. The selected pairs and high-count probabilities are (8, 9; 0.158), (18, 19; 0.603), and (77, 78; 0.451) for the three targets. Their aggregate accuracies are 0.260218, 0.295288, and 0.346181, equal to the adaptive accuracies to numerical precision.

Adaptive/fixed expected character counts at those matches are 2,967/4,136, 7,334/9,431, and 38,823/39,265. Thus adaptive character savings are 28.26% (95% interval 22.77–33.29%), 22.24% (12.15–31.17%), and 1.13% (–16.92–17.15%). Thus coding has a clear equal-accuracy cost advantage at the first two targets, while the data do not establish an advantage at 0.35. This diagnostic is deliberately non-implementable because its counts and mixing weights are selected from the final test curves. It affects only the comparator; Algorithm 3 remains one deterministic adaptive stopping rule.

4.4 Evaluation boundary

Alignment uses cached responses from 100 prompts per generator. Coding is conditioned on being solved by at least one of 512 cached programs, so it does not describe the excluded all-incorrect problems. Its development, validation, and final splits resample the same 83 problems rather than disjoint corpora. Coding utility uses final split IDs 75–84, and coding target quality uses IDs 95–104; selectors do not read their current final split. This is within-corpus repeated-split evidence, not evaluation on new identities. The utility multiplier and target profile are frozen on separate development splits; every test oracle is post-hoc.

5 Discussion

The theoretical and practical rules share one idea: continue while the value of another reward draw above the incumbent exceeds mean opening cost. The LLM implementations estimate both sides on a common scale. Alignment uses relative BT utility and a current-prompt tail, so its adaptive rule requires no training data. Coding uses a train-only isotonic correctness probability and one global upper-quarter tail. Response length represents unknown cost, is predicted by the observed-prefix mean without looking ahead, and is charged exactly after generation.

Train-selected Fixed- N is deployable; test-oracle Fixed- N removes transfer error in choosing one common count. The alignment character-budget oracle is stronger still: it knows prompt-specific mean lengths and allocates different counts. Adaptive stopping beats both Fixed- N baselines on average, whereas the cost-aware oracle exposes room to improve length prediction. Equal-cost alignment results separately show a quality advantage at fixed expected characters. In coding, adaptive stopping also outperforms the character-only budget selected on the training half.

Target attainment is harder: the divisor or integer N is selected before test, then quality moves with the held-out problems. Fixed- N transfers more accurately than the current alignment divisor grid. In coding, the frozen development inverse curve is close at all three targets but modestly overshoots the first and undershoots the other two. Cost savings therefore require calibration panels—without matched test quality they are not, by themselves, an efficiency comparison. The post-hoc equal-quality diagnostics remove that ambiguity. Alignment retains positive savings in every generator–target cell. Coding retains clear savings at targets 0.25 and 0.30, but its apparent advantage at 0.35 shrinks to 1.13% with an interval spanning zero.

The theorem does not certify these exponential tail fits; that would require finite-sample fair-cap concentration under reward-model error and length variation. The experiments instead test the fitted crossing rule end to end, providing empirical support and motivating a sharper joint analysis. The simplification study is nevertheless informative: eliminating the alignment prior and collapsing coding’s divisor-specific grid to one formula changes mean utility by only -0.00040 and -0.00057 , respectively, on the held-out confirmations.